

מבוא למדעי המחשב 67101 – סמסטר ב 2026

תרגיל 4 – Battleships

להגשה בתאריך **10/05/2026** בשעה 22:00

0. הקדמה - משחק צוללות (Battleships)

בתרגיל זה תממשו וריאציה של המשחק "צוללות". זהו משחק המיועד לשני שחקנים, לכל שחקן יש לוח עליו הוא ממקם בתחילת המשחק צוללות באורכים שונים במיקומים שונים בלוח. כל שחקן אינו רואה את הצוללות שהיריב הציב אלא אם פגע בהן במהלך המשחק. המטרה של כל שחקן היא להטביע את כל הצוללות בלוח של היריב.

לכל שחקן יש תור לסירוגין. בכל תור, השחקן בוחר מיקום בלוח של היריב אליו הוא יכול לשלוח "פצצה", ואם הוא פגע במיקום בו יש צוללת של היריב, הצוללת באותו מיקום מתגלה. כאשר כל אורך הצוללת מתגלה, הצוללת "טובעת" וכך השחקן מתקדם לקראת ניצחון.

השחקן שהצליח להטביע את כל הצוללות של היריב ראשון, הוא המנצח. ניתן לקרוא עוד על המשחק באתר

ויקיפדיה: [צוללות \(משחק\) – ויקיפדיה](#)

בגרסת המשחק אותה נממש בתרגיל זה יתקיימו התנאים הבאים:

1. את הצוללות ניתן להציב במאונך בלבד.
2. לצוללות מותר להיות צמודות לצוללות אחרות, אך כמו במשחק האמיתי, גם בתרגיל שלנו – לצוללות אסור לחפוף אחת לשנייה (משמע, צוללת אחת לא יכולה לחלוק משבצת עם צוללת אחרת).
3. במשחק יהיה שחקן אחד אנושי, שהוא המשתמש של המשחק, והוא ישחק מול שחקן אוטומטי של המחשב.

אנו ממליצים מאוד לקרוא את כל הוראות התרגיל המפורטות במסמך זה לפחות פעם אחת לפני שאתן

ניגשות לפתרון!

בנוסף, הקפידו על כתיבת קוד קריא על פי חוקי ה-Coding Style שהוצגו בקורס!

תוכן העניינים:

1. הקוד המסופק - תיאור קצר של הקבועים והמתודות בקוד המסופק עם התרגיל. ניתן להוריד את הקוד מהמודל בסמוך להוראות התרגיל.
2. ייצוג לוח המשחק - בחלק זה נסביר על אופן ייצוג לוח משחק בקוד.
3. הוראות למימוש התרגיל:

3.1. בחלק זה ניתן הנחיות מודרכות למימוש פונקציות האתחול של המשחק הכולל לוח לשחקן

האנושי, ולוח לשחקן האוטומטי.

3.2. בחלק זה ניתן הנחיות כלליות למהלך המשחק עצמו ואתם תצטרכו להחליט איך לממש את

המשחק בקוד.

4. הוראות הגשה.

1. הקוד המסופק :

• הקובץ helper.py

כדי להקל על התהליך, מסופק לכן קובץ עזר - **helper.py** המכיל קבועים שונים ופונקציות עזר שונות. עליכן לייבא (באמצעות import) את הקובץ לתוך הקובץ **battleship.py** כדי שתוכלו להשתמש בקבועים ובפונקציות המוגדרות בו.

שימו לב - יש להשתמש ישירות בקבועים המוגדרים בקובץ זה ולא להעתיק אותם לקוד שלכן!

מותר להגדיר קבועים נוספים אך עליהם להימצא בקובץ **battleship.py** בלבד ועליכן להשתמש בקבועים המסופקים לכן בקובץ **helper.py** במקומות הנדרשים לכך.

קבועים:

הקבועים המוגדרים בקובץ helper.py (שעליכן להשתמש בהם - ישנם עוד קבועים שאין צורך להשתמש בהם) הם:

1. NUM_ROWS: קבוע שהוא מספר שלם המייצג את מספר השורות בלוח המשחק.
2. NUM_COLUMNS: קבוע שהוא מספר שלם המייצג את מספר העמודות בלוח המשחק.
3. SHIP_SIZES: קבוע מסוג tuple המכיל את גדלי הצוללות (שימו לב שיכולות להיות צוללות שונות באותו גודל).
4. WATER: קבוע המייצג תא בלוח המשחק שיש בו מים (= תא ללא צוללת = תא ריק. כלומר – תא שיש בו מים זה כמו לומר תא ריק).
5. SHIP: קבוע המייצג תא בלוח שיש בו צוללת.
6. HIT_WATER: קבוע המייצג תא בלוח המשחק שהכיל מים ונפגע.
7. HIT_SHIP: קבוע המייצג תא בלוח המשחק שהכיל צוללת ונפגע.

פונקציות:

הפונקציות המוגדרות בקובץ **helper.py** (שעליכן להשתמש בהן - ישנן עוד פונקציות שאין צורך להשתמש בהן) הן:

1. **show_board(board1, board2=None):**

פונקציה זו מקבלת לוח אחד או שני לוחות ומדפיסה אותם על המסך.
אם מסופק רק לוח אחד - הפונקציה מדפיסה רק אותו. אם מסופקים שני לוחות - הפונקציה תדפיס את שניהם.
שימו לב - הפונקציה תדפיס את הלוח במלואו כפי שהוא מתקבל.

2. `get_input(msg):`

פונקציה זו מקבלת הודעה להדפסה, מדפיסה הודעה זו, מחכה לקבלת קלט מהמשתמש ומחזירה את הקלט שהתקבל.

שימו לב:

- יש להשתמש אך ורק בפונקציה זו כשאתן מבקשות קלט מהמשתמש בתרגיל, אין להשתמש ב `input()` מחוץ לה.

3. `show_msg(msg):`

פונקציה זו מדפיסה את ההודעה `msg` למסך.

שימו לב:

- אין להשתמש בפונקציה `print` כלל במהלך התוכנית!
- כל הדפסת פלט תעשה רק ע"י הפונקציה `show_msg` או הפונקציות `show_board/get_input`.

4. `is_cell_name(str):`

פונקציה זו מקבלת מחרוזת ובודקת שהערך שניתן מתאים לפורמט של `NX` המייצג תא בלוח המשחק, כך ש `N` היא אות, ו-`X` הוא מספר שלם. אם הפורמט מתאים - הפונקציה מחזירה `True`. אם לא - הפונקציה מחזירה `False`.

5. `choose_ship_location(board, size, locations):`

פונקציה זו מיועדת עבור השחקן האוטומטי שנממש בקוד. הפונקציה מקבלת לוח משחק, גודל של צוללת וקבוצת מיקומים אפשריים חוקיים (מהצורה - `((row_index, column_index))`) להצבת ראש הצוללת, ומחזירה מיקום אחד להצבת הצוללת (המיקום נבחר בצורה רנדומלית).
מיקום חוקי יהיה כל מיקום בגבולות הלוח כך שהצבת ראש הצוללת במיקום זה, יגרום לכך שכל חלקי הצוללת יוצבו במיקומים עם במים פנויים (כלומר מכיל את הערך `WATER`) ובגבולות הלוח.
הפונקציה יוצאת מנקודת הנחה שכל הערכים הנשלחים אליה חוקיים.
סוג המשתנה של הפרמטר `locations` הוא `set`.

6. `choose_torpedo_target(board, locations):`

גם פונקציה זו מיועדת עבור השחקן האוטומטי שנממש בקוד. הפונקציה מקבלת לוח (עם ספינות מוסתרות) וקבוצת מיקומים אפשריים (מהצורה - `((row_index, column_index))`) לשם שליחת פצצה (טורפדו) לצוללת של היריב ומחזירה מיקום לשליחת טורפדו (המיקום נבחר בצורה רנדומלית).

• הקובץ: battleship.py

הקובץ אותו אתם נדרשים להגיש. בקובץ המסופק יש חתימות של הפונקציות אותן אתם נדרשים לממש לפי הוראות התרגיל.

2. ייצוג לוח המשחק

לוחות המשחק מיוצגים ע"י רשימה דו מימדית.

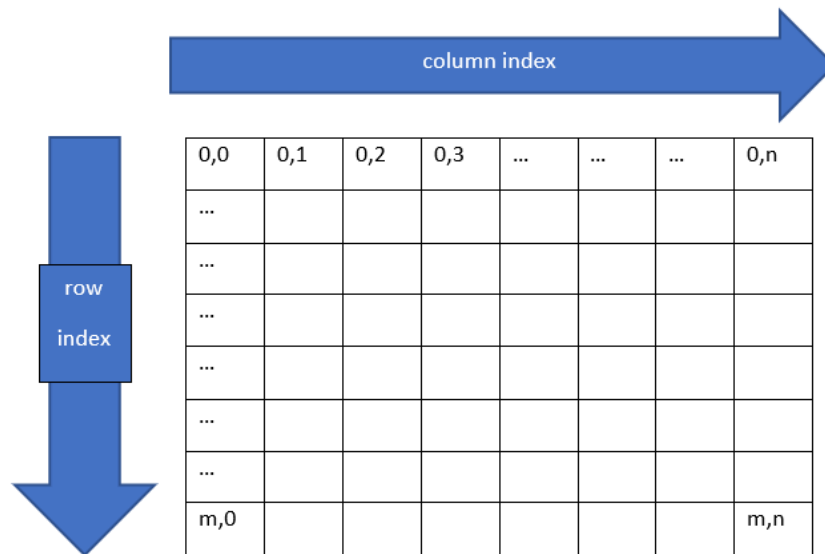
כל תא ברשימה דו מימדית (לאחר אתחול הרשימה) מייצג אלמנט כלשהו בלוח:

- מיקום של צוללת מיוצג ע"י הקבוע SHIP (מיוצג עם צבע חום בלוח).
- מיקום של פגיעה מיוצג ע"י הקבוע HIT_SHIP (מיוצג עם צבע אדום בלוח).
- מיקום של החטאה מיוצג ע"י הקבוע HIT_WATER (מיוצג עם צבע טורקיז בלוח).
- מיקום ריק (ללא צוללת או מוסתר כאשר מציגים את הלוח לשחק השני) מיוצג ע"י הקבוע WATER (מיוצג עם צבע כחול בלוח).

הערות:

- בכל מקום בו מצויין "**לוח המשחק**" (או **לוח**) הכוונה היא לרשימה דו מימדית מלבנית המייצגת את לוח המשחק.
- בכל מקום בו מצויין "**מיקום**" או "**קואורדינטה**" הכוונה היא ל-tuple המכיל זוג מספרים שלמים המהווים אינדקס של שורה ואינדקס של עמודה (בהתאמה) ברשימה דו מימדית המייצגת את הלוח:
(row_index, column_index) שימו לב שהאינדקס של השורה יבוא **לפני** האינדקס של העמודה.
לדוגמא: (5,3) הוא התא הנמצא בשורה השישית ועמודה הרביעית (מפני שאינדקסים בפייתון מתחילים מ-0. 5 הוא האינדקס הראשון ולכן מייצג את השורה ה-5+1, ובאופן דומה 3 הוא האינדקס השני ולכן מייצג את העמודה ה-3+1).
- ייצוג הצבעים הוא פנימי לקובץ העזר, **אין להשתמש/להוסיף/לשנות אותו.**

- דוגמא לייצוג של רשימה דו מימדית בעלת m שורות ו- n עמודות:



- בעוד אנחנו עובדים בקוד עצמו עם מטריצה דו מימדית בעלת אינדקסים מספריים, לוח המשחק המודפס לשחקן מייצג מיקום ע"י אינדקס מספרי לשורות, ואינדקס לקסיקלי (מבוסס אותיות) עבור העמודות.

לדוגמא:

	A	B	C	D	E	F	G	H
1	.	.	.	.	.	.	.	.
2	.	.	.	.	.	.	.	.
3	.	.	.	.	.	.	.	.
4	.	.	.	.	.	.	.	.
5	.	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.
9	.	.	.	.	.	.	.	.
10	.	.	.	.	.	.	.	.

- שימו לב שהצבעים יודפסו רק בהרצה דרך הטרמינל.

3. הוראות מימוש:

3.1 אתחול משחק:

נתחיל במימוש פונקציות לצורך אתחול המשחק. חלק זה של התרגיל יחסית מודרך, ויוגדרו עבורכם אילו פונקציות ליצור. כמובן שאלו לא הפונקציות היחידות שאתם תיצרו בחלק זה, ואנחנו מעודדים אתכם ליצור פונקציות עזר בהן הפונקציות שהגדרנו ישתמשו.

נממש פונקציות לצורך (1) יצירת לוח ריק, (2) בדיקה האם מיקום בלוח הוא חוקי להכנסת צוללת, (3) המרת קלט של המשתמש לקואורדינטות בלוח, (4) אתחול לוח של השחקן האנושי, (5) אתחול לוח של השחקן האוטומטי.

בכל הפונקציות ניתן להניח שהמספרים המייצגים גדלים המתקבלים כפרמטרים לפונקציות הם מספרים חיוביים.

3.1.1 ממשו את הפונקציה:

`init_board(rows, columns)`

הפונקציה צריכה לקבל שני פרמטרים:

a. rows - מספר השורות בלוח.

b. columns - מספר העמודות בלוח.

על הפונקציה ליצור לוח ריק בגודל השורות כפול העמודות עבור אחד השחקנים - לוח המלא כולו בקבוע

WATER המסמל מיקום ריק. הפונקציה תחזיר את הלוח.

שימו לב: הרשימה המוחזרת היא רשימה של רשימות בעלות ערכים זהים, אבל יש לוודא בעת היצירה שלה שהרשימות הן רשימות נפרדות.

3.1.2 ממשו את הפונקציה:

`valid_ship(board, size, loc)`

הפונקציה צריכה לקבל שלושה פרמטרים:

a. board - לוח משחק

b. size - גודל ספינה

c. loc - משתנה מסוג tuple שמייצג מיקום בלוח עבור ראש הצוללת **מהצורה** -

`(row_index, column_index)`. ניתן להניח שהערכים הם מספרים, אך לא ניתן להניח שהם

בגבולות הלוח.

הפונקציה תבדוק האם צוללת בגודל size שמתחילה בקואורדינטה **(מהצורה)**

`(row_index, column_index)` יכולה להיכנס ללוח **במאונך** ללא התנגשות בקצות הלוח או בצוללת אחרת

שכבר נמצאת בלוח.

הפונקציה תחזיר True אם הצוללת יכולה להיכנס ללוח באופן חוקי, ו-False אחרת.

שימו לב שפונקציה זו לא מבצעת שינויים בלוח, רק מבצעת בדיקה.

לדוגמא, ניתן להכניס צוללת בגודל 5 בקואורדינטה 1A באופן הבא:

	A	B	C	D	E	F	G	H
1	x	.	.	.	.	.	.	.
2	x	.	.	.	.	.	.	.
3	x	.	.	.	.	.	.	.
4	x	.	.	.	.	.	.	.
5	x	.	.	.	.	.	.	.
6	.	.	.	.	.	.	.	.
7	.	.	.	.	.	.	.	.
8	.	.	.	.	.	.	.	.
9	.	.	.	.	.	.	.	.
10	.	.	.	.	.	.	.	.

שימו לב שבדוגמא זו שלא נוכל להכניס עוד צוללת בקואורדינטה 2A כיוון שזו תתנגש עם הצוללת הראשונה שהוכנסה.

לא נוכל גם להכניס צוללת בגודל 5 בקואורדינטה 9A בלוח זה, כיוון שזו תגרום ליציאה מקצות הלוח.

3.1.3. ממשו את הפונקציה:

`cell_name_to_loc(name)`

הפונקציה מקבלת פרמטר יחיד בשם `name`, שמייצג קלט טקסטואלי מהמשתמש המתאר מיקום כלשהו בלוח המודפס ומחזירה tuple המתאר קואורדינטה מספרית.

הקלט שהשחקן מספק יהיה מהפורמט XN כאשר X מתארת עמודה כלשהי המיוצגת ע"י אות אנגלית גדולה, ו-N מתאר את מספר השורה. **לדוגמא:** אם המשתמש סיפק את הקלט "2A" הפונקציה תחזיר את הערך (1,0) שכן העמודה A היא העמודה הראשונה (כלומר באינדקס 0) ושורה 2 היא השורה באינדקס 1.

שימו לב!

- קלט חוקי לפונקציה הוא רק **אות גדולה ומספר** בפורמט XN.
- ניתן להניח שהערך שהתקבל הוא מסוג מחרוזת ובפורמט החוקי (כלומר תו ולאחריו מספר שלם).
- **הקלט הפוך מייצוג קואורדינטה.** בקלט הטקסטואלי מופיעה קודם העמודה ואז השורה, ואילו בייצוג של קואורדינטה מופיע קודם האינדקס של השורה ואז האינדקס של העמודה.
- מספר השורות בלוח המודפס מתחיל ב-1 ולא ב-0!
- ניתן להניח שיהיו לכל היותר 26 עמודות.
- לא ניתן להניח שהערך מייצג קואורדינטה בגבולות הלוח של המשחק (**הפונק' לא מקבלת לוח כקלט ולכן גם לא ניתן לבדוק זאת**).

=====

טיפ:

בשביל לבצע את ההמרה בין הייצוג הטקסטואלי של אות לייצוג המספרי שלה ניתן להשתמש בפונקציה `ord`

(ניתן לקרוא עוד על הפונקציה ord [באן](#)).

הפונקציה ord מקבלת מחרוזת באורך 1 המייצגת אות ומחזירה ערך מספרי עבור האות הזו לפי ייצוג ASCII. למשל הקוד:

ord("A")

יחזיר את הערך 65, בעוד הקוד:

ord("D")

יחזיר את הערך 68.

בשביל לבצע המרה חזרה מערך ASCII מספרי לייצוג טקסטואלי ניתן להשתמש בפונקציה chr (ניתן לקרוא עוד על הפונקציה chr [באן](#)).

למשל הקוד:

chr(68)

יחזיר את הערך "D".

=====

3.1.4. ממשו את הפונקציה:

create_player_board(rows, columns, ship_sizes)

הפונקציה צריכה לקבל שלושה פרמטרים:

a. rows – מספר השורות בלוח.

b. columns - מספר העמודות בלוח.

c. ship_sizes – רשימה או טופל (tuple) של מספרים שלמים שמתארים את אורכי הצוללות שיופיעו בלוח.

הפונקציה תייצר לוח ריק חדש עבור השחקן, ועבור כל גודל צוללת הנמצא ברשימה ship_sizes, הפונקציה תציב צוללת בלוח על פי הקואורדינטות שהמשתמש יכניס.

הפונקציה תפעל כך:

1. הפונקציה תיצור לוח ריק

2. כל עוד נותרו ספינות שצריך למקם :

2.1. הפונקציה תדפיס את מצב הלוח הנוכחי (עם הפונקציה show_board).

2.2. הפונקציה תבקש מהמשתמש מיקום להנחת הספינה הבאה.

● הקלט מהמשתמש יתקבל בעזרת הפונקציה get_input שמומשה עבורכם

בקובץ helper.py

● הפורמט בו הקלט צריך להתקבל הוא: **XN** כאשר X היא אות (גדולה או קטנה)

המייצגת טור ו-N הוא מספר שלם המייצג שורה.

- הקוד צריך לתמוך בכך שהאות שתתקבל מהמשתמש יכולה להיות אות גדולה או קטנה (משמע, שניהם ערכים חוקיים) אך שימו לב שבקלט של הפונקציה cell_name_to_loc ערך חוקי יהיה רק עם אות גדולה בלבד.

- יש להציג עם הבקשה הודעה מתאימה כדי שהשחקן יבין מה הוא אמור לעשות.

2.3. **הפונקציה תבדוק אם הפורמט של הקלט תקין** (בעזרת פונקציית העזר המסופקת לכם).

2.4. **אם הפורמט תקין**, הפונקציה תמיר את הקלט לקואורדינטות של הלוח. (שימו לב שכבר מימשתם פונקציית המרה כזו).

2.4.1. אם ניתן למקם את הצוללת על הלוח במיקום שהתקבל, הצוללת תמוקם על

הלוח כאשר הקואורדינטה שניתנה היא הקואורדינטה העליונה ביותר (ראו

דוגמא). עדכנו את הלוח והמשיכו להשמת הצוללת הבאה (2.1).

2.5. **אם לא ניתן להמיר או שלא ניתן למקם צוללת במיקום התקבל**, הדפיסו הודעת שגיאה

אינפורמטיבית (שיכולה להיות כל דבר, העיקר שתסמן שהקלט אינו חוקי) וחזרו ל-2.1.

3. **בסוף הפונקציה יש להחזיר את לוח המשחק.**

הערות:

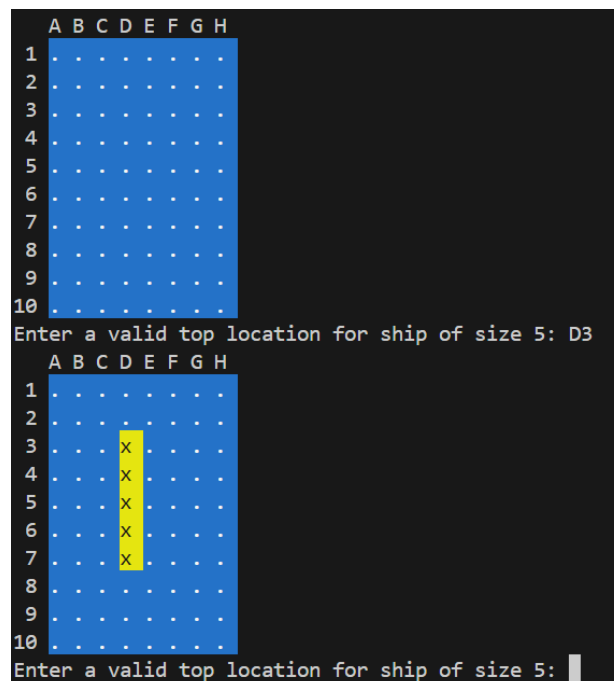
a. ב-2.3 הקוד צריך להתמודד עם כל קלט שהמשתמש מקליד אך לא נתקדם ל-2.4 כל עוד הקלט לא בפורמט החוקי.

b. ניתן להניח שניתן יהיה להכניס את כל הספינות ברשימה שהתקבלה ללוח.

c. המשחק אמנם לא יהיה מעניין, אך גם רשימה ריקה היא קלט חוקי.

דוגמאות הרצה:

- הכנסה חוקית:



- הכנסה לא חוקית (קלט הגורם להתנגשות עם צוללת אחרת, ועל כן הלוח לא השתנה):

```

Enter a valid top location for ship of size 5: D3
A B C D E F G H
1 . . . . .
2 . . . . .
3 . . . X .
4 . . . X .
5 . . . X .
6 . . . X .
7 . . . X .
8 . . . . .
9 . . . . .
10 . . . . .
Enter a valid top location for ship of size 5: D4
Location not valid!
A B C D E F G H
1 . . . . .
2 . . . . .
3 . . . X .
4 . . . X .
5 . . . X .
6 . . . X .
7 . . . X .
8 . . . . .
9 . . . . .
10 . . . . .
Enter a valid top location for ship of size 5: 

```

- הכנסה לא חוקית (קלט שלא נמצא בפורמט, על כן הלוח לא ישתנה):

```

A B C D E F G H
1 . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
5 . . . . .
6 . . . . .
7 . . . . .
8 . . . . .
9 . . . . .
10 . . . . .
Enter a valid top location for ship of size 5: Da
Location not valid!
A B C D E F G H
1 . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
5 . . . . .
6 . . . . .
7 . . . . .
8 . . . . .
9 . . . . .
10 . . . . .
Enter a valid top location for ship of size 5: 

```

- הכנסה לא חוקית (מיקום לא חוקי):

```

A B C D E F G H
1 . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
5 . . . . .
6 . . . . .
7 . . . . .
8 . . . . .
9 . . . . .
10 . . . . .
Enter a valid top location for ship of size 5: J0
Location not valid!
A B C D E F G H
1 . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
5 . . . . .
6 . . . . .
7 . . . . .
8 . . . . .
9 . . . . .
10 . . . . .
Enter a valid top location for ship of size 5: 

```

3.1.5 ממשו את הפונקציה

create_computer_board(rows, columns, ship_sizes):

תפקיד פונקציה זו הוא ליצור את הלוח עבור השחקן האוטומטי. באופן דומה לפונקציה היוצרת את לוח השחקן האנושי, הפונקציה צריכה לקבל שלושה פרמטרים:

- d. rows – מספר השורות בלוח.
- e. columns – מספר העמודות בלוח.
- f. ship_sizes – רשימה או טופל (tuple) של מספרים שלמים שמתארים את אורכי הצוללות שיופיעו בלוח.

הפונקציה תייצר לוח ריק חדש עבור השחקן, ועבור כל גודל צוללת הנמצא ברשימה ship_sizes, הפונקציה תציב צוללת בלוח בצורה רנדומלית.

את אופן מימוש הפונקציה הזו אנחנו משאירים לכם, אך אתם מחויבים להשתמש במתודה choose_ship_location מקובץ העזר עבור בחירת מיקום חוקי רנדומלי להצבת צוללת על הלוח.

3.2 הרצת המשחק:

בחלק זה של התרגיל מלבד הפונקציה `main` לא נדריך אתכם אילו פונקציות ליצור, אלא נתאר בקווים כללים איך המשחק אמור להתנהל ועליכם לחשוב בעצמכם אילו פונקציות עליכם ליצור כדי לבנות את הקוד בצורה טובה.

ממשו את הפונקציה:

`main()`

1. הפונקציה תיצור את הלוחות שימוקמו בהן הצוללות עבור השחקן האנושי ועבור השחקן האוטומטי בעזרת הפונקציות שמימשותם בחלק הקודם של התרגיל (ופונקציות נוספות אם יש בכך צורך), ולאחר מכן תתחיל את המשחק עצמו.

2. הפונקציה אחראית על הפעלת התורים במשחק. סיבוב אחד (מהלך שני תורים במשחק - אחד של השחקן האנושי ואחד של השחקן האוטומטי) מתבצע באופן הבא:
כל עוד יש בשני הלוחות צוללות שעוד לא התגלו:

2.1. **הדפסת שני הלוחות** בעזרת הפונקציה `show_board` שמומשה עבורכם בקובץ `helper.py`.
הפונקציה מקבלת שני לוחות כפרמטרים כאשר הלוח שמתקבל בפרמטר הראשון הוא הלוח שהשחקן האנושי יצר שיודפס באופן מלא (נראה את מיקומי הצוללות) והלוח שמתקבל בפרמטר השני הוא לוח מוסתר של המחשב, בו הספינות החבויות יודפסו כמרים.
שימו לב! כדי להדפיס לוח מוסתר עליכם לדאוג לכך שהצוללות שעדיין לא התגלו על הלוח יוסתרו לפני שליחתו להדפסה.

2.2. **בקשת קלט עבור מטרת התקיפה של השחקן האנושי** - תודפס הודעה לשחקן האנושי שתבקש ממנו להכניס מיקום לשליחת פצצת טורפדו. השתמשו בפונקציה `get_input` שמומשה עבורכם בקובץ `helper.py`.

2.2.1. אם הקואורדינטה שהתקבלה תקינה והיא מסמנת תא בלוח שעדיין חבוי, נמשיך ל-2.3.
2.2.2. אם הקלט אינו תקין (פורמט לא חוקי או תא בלוח שאינו חבוי) הדפיסו הודעת שגיאה אינפורמטיבית וחזרו ל-2.2.

שימו לב:

- תוכן ההודעה לבחירתכם, העיקר שתצביע על כך שהקלט לא תקין
 - תזכורת אין להשתמש בפונקציה `print` אלא רק בפונקציות העזר של הקוד המסופק.
 - ניתן לקבל אותיות קטנות כקלט תקין.
- בשביל להמיר אותיות קטנות לגדולות ניתן להשתמש בפונקציה `upper` של `string` (עוד מידע על הפונקציה `upper` [בלינק](#)).

2.3. **הגרלת מטרה עבור השחקן האוטומטי**. השתמשו בפונקציה `choose_torpedo_target` הנתונה לכם בקובץ העזר כדי לבחור מיקום רנדומלי לשליחת פצצה מתוך רשימת מיקומים חוקיים.

2.4. **ביצוע הירי** - ביצוע ירי הפצצה ע"י 2 הצדדים ועדכון הלוח.

3. במקרה ולאחר פגיעה הושמד הצי של אחד השחקנים (או שניהם!), יודפסו שני הלוחות **בצורה גלויה**.

4. לאחר מכן תודפס שאלה אם לשחק שוב יחד עם הודעה על תוצאת המשחק שהסתיימה. הקלט החוקי

במקרה זה הוא "Y" או "ח", "Y" ("Y", "N") אינם ערכים חוקיים כאן).

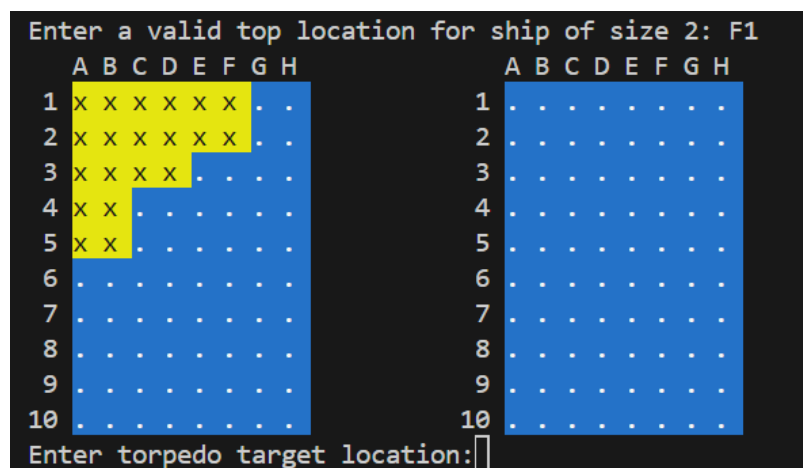
4.1. אם מתקבל קלט לא חוקי, יש להמשיך לשאול (עם הודעה מתאימה), אך ללא הדפסת הלוחות.

4.2. אם הקלט הוא "Y" יש להתחיל משחק חדש מנקודה 1.

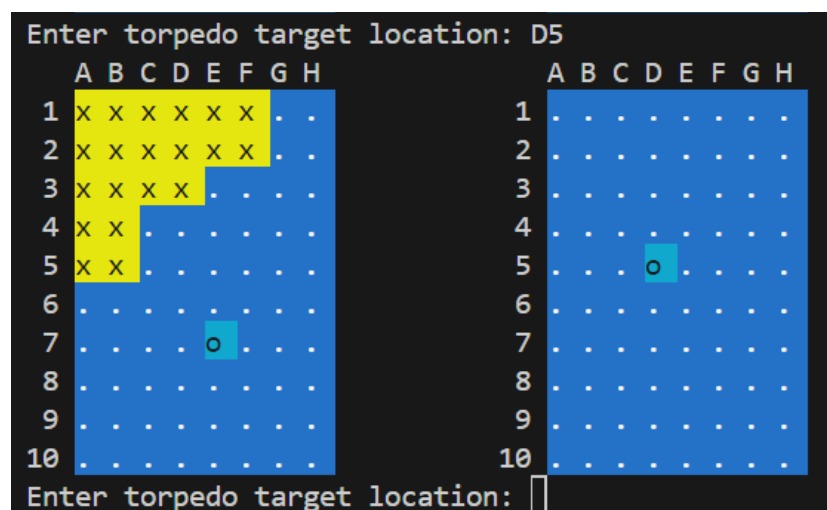
4.3. אם הקלט הוא "N" יש לסיים את התוכנית.

דוגמאות להרצת המשחק לאחר שלב האתחול:

- תחילת המשחק: בדוגמה הבאה אנו רואים את שני הלוחות לאחר שמיקמנו את כל הצוללות של השחקן האנושי. הלוח של השחקן - שלנו - הוא הלוח השמאלי וניתן לראות בו את הצוללות לאחר שמיקמנו את הצוללת האחרונה בגודל 2 במיקום F1. הלוח הימני הוא הלוח של השחקן האוטומטי- והצוללות בו לא נראות לנו:



- דוגמא לקלט חוקי: הקלט שהכנסנו הוא 5D ולכן אנחנו מנסים לפגוע בצוללת שנמצאת בקואורדינטה (4,3). הפגיעה לא הצליחה כי לא הייתה שם צוללת של היריב. השחקן האוטומטי בחר בקואורדינטה 7E שגם בה אין צוללת.



- דוגמא לקלט לא חוקי - הכנסנו בתור קלט את המחרוזת 90B שהוא קלט לא חוקי וקיבלנו הדפסה של
invalid target:

```

Enter torpedo target location: B90
Location not valid!
  A B C D E F G H
1 x x x x x x . .
2 x x x x x x . .
3 x x x x . . . .
4 x x . . . . . .
5 x x . . . . . .
6 . . . . . . . .
7 . . . . . . . .
8 . . . . . . . .
9 . . . . . . . .
10 . . . . . . . .
Enter torpedo target location: 

```

4. הוראות הגשה

- עליבן להגיש את הקובץ ex4.zip (בלבד) בקישור ההגשה של תרגיל 4 דרך אתר הקורס (moodle).
- ההגשה היא עד המועד הנקוב בראש הקובץ.
- ex4.zip צריך לכלול את הקובץ הבא בלבד:
battleship.py. o
- שימו לב שאין להגיש את הקובץ helper.py, ושהבדיקות יתבצעו עם קובץ עזר בעל ערכים שונים.

הנחיות כלליות בנוגע להגשה

- הנכם רשאים להגיש תרגילים דרך מערכת ההגשות באתר הקורס מספר רב של פעמים. ההגשה האחרונה בלבד היא זו שקובעת ושתיבדק.
- לאחר הגשת התרגיל, ניתן ומומלץ להוריד את התרגיל המוגש ולוודא כי הקבצים המוגשים הם אלו שהתכוונתם להגיש וכי הקוד עובד על פי ציפיותיכם.
- o באחריותכם לוודא כי – PDF הבדיקות נראה כמו שצריך.
- קראו היטב את קובץ נהלי הקורס לגבי הנחיות נוספות להגשת התרגילים.

בהצלחה!